Evaluación Arquitectónica bajo ISO/IEC 25010

VotaGT - Sistema de Gestión Electoral

Estándar de referencia: ISO/IEC 25010

Curso: Análisis de Sistemas II

Versión: 1.0

Fecha: Octubre 2026

---

1. Objetivo

El presente documento evalúa diferentes alternativas arquitectónicas para VotaGT utilizando atributos de calidad relacionados con ISO/IEC 25010.

Se comparan:

- Arquitectura en Capas.

- Repository Pattern.

- Microservicios.

- Arquitectura Orientada a Eventos (EDA).

Los atributos evaluados principalmente son:

- Mantenibilidad.

- Desempeño.

- Escalabilidad.

- Fiabilidad.

- Seguridad.

- Compatibilidad.

---

2. Contexto de VotaGT

VotaGT es un Sistema de Gestión Electoral que administra:

- Autenticación.

- Roles y permisos.

- Votaciones.

- Candidatos.

- Mesas.

- Resultados.

- Reportes.

- Auditoría.

Actualmente se utiliza:

- Node.js + Express.

- PostgreSQL.

- Redis.

- MongoDB.

- Neo4j.

La solución fue implementada como un Monolito Modular con API REST.

---

3. Atributos de calidad evaluados

3.1 Mantenibilidad

Capacidad del sistema para ser modificado, corregido o ampliado sin afectar innecesariamente otros componentes.

3.2 Desempeño

Capacidad del sistema para responder a las solicitudes utilizando adecuadamente los recursos disponibles.

3.3 Escalabilidad

Capacidad de soportar un incremento en usuarios, solicitudes o volumen de información.

3.4 Fiabilidad

Capacidad de continuar proporcionando las funciones esperadas ante condiciones normales o fallos parciales.

3.5 Seguridad

Capacidad de proteger la información y controlar el acceso a las funcionalidades.

3.6 Compatibilidad

Capacidad del sistema para comunicarse con otras aplicaciones y tecnologías.

---

4. Alternativa 1 - Arquitectura en Capas

Descripción

La aplicación se divide en responsabilidades como:

`Routes -> Controllers -> Services -> Repositories`

Mantenibilidad

Alta.

Cada capa tiene una responsabilidad definida, facilitando localizar cambios y corregir errores.

Desempeño

Bueno para el alcance actual.

Las llamadas entre capas se realizan dentro del mismo proceso Node.js, evitando comunicación de red entre módulos internos.

Escalabilidad

Media.

El sistema completo puede escalar como una unidad, pero sus módulos no se despliegan independientemente.

Fiabilidad

Media-Alta.

La simplicidad reduce puntos de fallo relacionados con comunicación distribuida.

Aplicación en VotaGT

Es una de las principales decisiones utilizadas en el backend actual.

---

5. Alternativa 2 - Repository Pattern

Descripción

Repository separa la lógica de negocio del acceso a datos.

Ejemplo:

```
ResultadoService -> ResultadoRepository -> PostgreSQL
```

Mantenibilidad

Alta.

Los cambios en consultas SQL quedan concentrados en los repositories.

Desempeño

Neutral.

El patrón no mejora directamente el rendimiento, pero permite optimizar el acceso a datos sin modificar los servicios.

Escalabilidad

Media-Alta.

Permite sustituir o adaptar mecanismos de persistencia con menor impacto sobre el dominio.

Fiabilidad

Alta para la organización interna.

Reduce el acoplamiento entre lógica de negocio y persistencia.

Aplicación en VotaGT

VotaGT implementa repositories para:

- Autenticación.

- Votaciones.

- Candidatos.

- Mesas.

- Resultados.

---

6. Alternativa 3 - Microservicios

Descripción

Cada capacidad principal podría convertirse en un servicio independiente.

Por ejemplo:

- Auth Service.

- Election Service.

- Result Service.

- Audit Service.

Mantenibilidad

Alta cuando los límites de dominio están correctamente definidos.

Sin embargo, aumenta la complejidad operativa.

Desempeño

Puede disminuir debido a comunicación entre servicios mediante red.

Requiere mecanismos adicionales para reducir latencia.

Escalabilidad

Muy alta.

Cada servicio puede escalar de forma independiente.

Fiabilidad

Puede ser alta, pero requiere:

- Circuit Breakers.

- Retries.

- Observabilidad.

- Balanceadores.

- Gestión de fallos distribuidos.

Desventajas para VotaGT

Para el alcance académico actual introduciría complejidad innecesaria relacionada con:

- Despliegue.

- Seguridad distribuida.

- Comunicación remota.

- Consistencia.

- Monitoreo.

- DevOps.

Decisión

No se selecciona como arquitectura actual.

Puede considerarse una evolución futura si el volumen del sistema aumenta significativamente.

---

7. Alternativa 4 - Arquitectura Orientada a Eventos (EDA)

Descripción

Los componentes reaccionan ante eventos generados por otros componentes.

Ejemplo implementado:

`ResultadoService -> EventBus -> AuditObserver -> MongoDB`

Mantenibilidad

Alta cuando se utiliza para funciones desacopladas.

El emisor del evento no necesita conocer directamente al consumidor.

Desempeño

Bueno para operaciones que pueden ejecutarse de forma desacoplada.

Escalabilidad

Alta.

Los consumidores de eventos pueden evolucionar independientemente.

Fiabilidad

Depende del mecanismo de entrega.

La implementación actual mediante EventEmitter proporciona desacoplamiento, aunque no constituye una cola durable de eventos.

Aplicación en VotaGT

Se utiliza EDA de manera parcial para la auditoría mediante Observer Pattern.

Eventos actuales:

- `LOGIN\_EXITOSO`

- `VOTACION\_CREADA`

- `CANDIDATO\_CREADO`

- `MESA\_CREADA`

- `RESULTADO\_REGISTRADO`

---

8. Matriz comparativa

| Alternativa | Mantenibilidad | Desempeño | Escalabilidad | Complejidad | Aplicación actual |

|---|---|---|---|---|---|

| Arquitectura en Capas | Alta | Alta | Media | Baja | Sí |

| Repository | Alta | Media-Alta | Media-Alta | Baja | Sí |

| Microservicios | Alta | Media | Muy alta | Muy alta | No |

| EDA | Alta | Alta para procesos desacoplados | Alta | Media | Parcial |

---

9. Escenarios de calidad

ESC-CAL-001 - Mantenibilidad

Fuente: Equipo de desarrollo.

Estímulo: Se requiere modificar la lógica de validación de candidatos.

Entorno: Sistema en mantenimiento.

Artefacto: `candidato.service.js`.

Respuesta: El cambio deberá realizarse principalmente dentro del módulo de candidatos sin modificar los módulos de mesas o resultados.

Medida: El cambio deberá permanecer localizado en el componente responsable y sus pruebas relacionadas.

ESC-CAL-002 - Desempeño

Fuente: Usuario autenticado.

Estímulo: Solicita repetidamente los resultados de una votación.

Entorno: Operación normal.

Artefacto: ResultadoService / Redis.

Respuesta: Después de una primera consulta, los resultados deberán poder recuperarse desde Redis mientras la caché permanezca vigente.

Medida: Se deberá evidenciar:

```
CACHE MISS -> CACHE GUARDADA -> CACHE HIT
```

ESC-CAL-003 - Fiabilidad

Fuente: Redis.

Estímulo: Redis deja de estar disponible.

Entorno: API en ejecución.

Artefacto: Cache Gateway / Circuit Breaker.

Respuesta: La aplicación deberá continuar obteniendo resultados desde PostgreSQL.

Medida: La API deberá continuar respondiendo HTTP 200 aunque Redis se encuentre en estado degradado.

ESC-CAL-004 - Recuperación

Fuente: Redis.

Estímulo: Redis vuelve a estar disponible después de una falla.

Entorno: Circuit Breaker en estado OPEN.

Artefacto: Circuit Breaker.

Respuesta: Después del periodo configurado se realizará un intento de recuperación.

Medida: Se deberá observar la transición:

`OPEN -> HALF\_OPEN -> CLOSED`

ESC-CAL-005 - Seguridad

Fuente: Usuario no autenticado.

Estímulo: Intenta acceder a un endpoint protegido.

Entorno: Operación normal.

Artefacto: Auth Middleware.

Respuesta: El sistema deberá impedir el acceso.

Medida: HTTP 401 con `application/problem+json`.

ESC-CAL-006 - Autorización

Fuente: Usuario autenticado sin permiso.

Estímulo: Intenta realizar una operación restringida.

Entorno: Operación normal.

Artefacto: Permission Middleware.

Respuesta: La operación deberá ser rechazada.

Medida: HTTP 403.

ESC-CAL-007 - Interoperabilidad

Fuente: Cliente externo.

Estímulo: Consume la API REST.

Entorno: API disponible.

Artefacto: OpenAPI / Express.

Respuesta: El cliente deberá poder comunicarse mediante HTTP y JSON utilizando el contrato publicado.

Medida: Los endpoints deberán ser reproducibles desde Postman o Swagger.

---

10. Evaluación de la arquitectura seleccionada

Para el alcance actual de VotaGT se selecciona:

Monolito Modular + Arquitectura en Capas + Repository + componentes orientados a eventos

Esta combinación ofrece un equilibrio adecuado entre:

- Mantenibilidad.

- Rendimiento.

- Simplicidad.

- Capacidad de prueba.

- Tolerancia a fallos.

- Escalabilidad futura.

No se selecciona una arquitectura completa de microservicios debido al aumento de complejidad que introduciría en el alcance actual.

---

11. Relación con mecanismos implementados

| Atributo de calidad | Mecanismo |

|---|---|

| Mantenibilidad | Capas, Services, Repository |

| Desempeño | Redis y Cache-Aside |

| Fiabilidad | Circuit Breaker y fallback PostgreSQL |

| Seguridad | JWT y permisos |

| Auditabilidad | Observer + MongoDB |

| Interoperabilidad | API REST + OpenAPI 3.0.3 |

| Capacidad de prueba | node:test, Postman y BDD |

| Modularidad | Monolito Modular |

---

12. Conclusión

La evaluación muestra que la arquitectura seleccionada proporciona una solución adecuada para el alcance actual de VotaGT.

La arquitectura en capas y Repository favorecen la mantenibilidad; Redis mejora el desempeño; Circuit Breaker mejora la tolerancia a fallos; Observer permite desacoplar la auditoría; y la API REST facilita la interoperabilidad.

Una arquitectura basada completamente en microservicios podría ofrecer mayor escalabilidad independiente, pero actualmente introduciría una complejidad superior a las necesidades del proyecto.